

Recursive Algorithms

Section 5.4

Section Summary

- Recursive Algorithms
- Proving Recursive Algorithms Correct
- Recursion and Iteration
- Merge Sort

Recursive Algorithms

Definition: An algorithm is called *recursive* if it solves a problem by reducing it to an instance of the same problem with smaller input.

- For the algorithm to terminate, the instance of the problem must eventually be reduced to some initial case for which the solution is known.

Recursive Factorial Algorithm

Example: Give a recursive algorithm for computing $n!$, where n is a nonnegative integer.

- **Solution:** Use the recursive definition of the factorial function.

```
procedure factorial( $n$ : nonnegative integer)
  if  $n = 0$  then return 1
  else return  $n \cdot \text{factorial}(n - 1)$ 
  {output is  $n!$ }
```

Recursive Exponentiation Algorithm

Example: Give a recursive algorithm for computing a^n , where a is a nonzero real number and n is a nonnegative integer.

Solution: Use the recursive definition of a^n .

```

procedure power( $a$ : nonzero real number,  $n$ : nonnegative
integer)
if  $n = 0$  then return 1
else return  $a \cdot \text{power}(a, n - 1)$ 
{output is  $a^n$ }
  
```

Recursive GCD Algorithm

Example: Give a recursive algorithm for computing the greatest common divisor of two nonnegative integers a and b with $a < b$.

Solution: Use the reduction

$$\gcd(a, b) = \gcd(b \bmod a, a)$$

and the condition $\gcd(0, b) = b$ when $b > 0$.

```

procedure gcd( $a, b$ : nonnegative integers
with  $a < b$ )
if  $a = 0$  then return  $b$ 
else return gcd( $b \bmod a, a$ )
{output is  $\gcd(a, b)$ }
  
```

- Let m be a positive integer and let a and b be integers. Then

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$
and

$$ab \bmod m = ((a \bmod m) (b \bmod m)) \bmod m.$$

Recursive Modular Exponentiation Algorithm

Example: Devise a recursive algorithm for computing $b^n \bmod m$, where b , n , and m are integers with $m \geq 2$, $n \geq 0$, and $1 \leq b \leq m$.

- Solution:** (see text for full explanation)

```

procedure mpower( $b, n, m$ : integers with  $b > 0$  and  $m \geq 2, n \geq 0$ )
if  $n = 0$  then
    return 1
else if  $n$  is even then
    return  $\text{mpower}(b, n/2, m)^2 \bmod m$ 
else
    return  $(\text{mpower}(b, \lfloor n/2 \rfloor, m)^2 \bmod m \cdot b \bmod m) \bmod m$ 
{output is  $b^n \bmod m$ }
  
```

Recursive Binary Search Algorithm

Example: Construct a recursive version of a binary search algorithm.

Solution: Assume we have $a_1, a_2, \dots, a_n$, an increasing sequence of integers. Initially i is 1 and j is n . We are searching for x .

```

procedure binary search( $i, j, x$  : integers,  $1 \leq i \leq j \leq n$ )
 $m := \lfloor (i + j) / 2 \rfloor$ 
if  $x = a_m$  then
    return  $m$ 
else if  $(x < a_m \text{ and } i < m)$  then
    return binary search( $i, m-1, x$ )
else if  $(x > a_m \text{ and } j > m)$  then
    return binary search( $m+1, j, x$ )
else return 0
{output is location of  $x$  in  $a_1, a_2, \dots, a_n$  if it appears, otherwise 0}
  
```

Proving Recursive Algorithms Correct

- Both mathematical and strong induction are useful techniques to show that recursive algorithms always produce the correct output.

Example: Prove that the algorithm for computing the powers of real numbers is correct.

```

procedure power( $a$ : nonzero real number,  $n$ : nonnegative integer)
if  $n = 0$  then return 1
else return  $a \cdot \text{power}(a, n-1)$ 
{output is  $a^n$ }
  
```

Solution: Use mathematical induction on the exponent n .

BASIS STEP: $a^0 = 1$ for every nonzero real number a , and $\text{power}(a, 0) = 1$.

INDUCTIVE STEP: The inductive hypothesis is that $\text{power}(a, k) = a^k$, for all $a \neq 0$.

Assuming the inductive hypothesis, the algorithm correctly computes a^{k+1} , since

$$\text{power}(a, k+1) = a \cdot \text{power}(a, k) = a \cdot a^k = a^{k+1}.$$



Merge Sort (归并排序)

- *Merge Sort* works by iteratively splitting a list (with an even number of elements) into two sublists of equal length until each sublist has one element.
- Each sublist is represented by a balanced binary tree.
- At each step a pair of sublists is successively merged into a list with the elements in increasing order. The process ends when all the sublists have been merged.
- The succession of merged lists is represented by a binary tree.

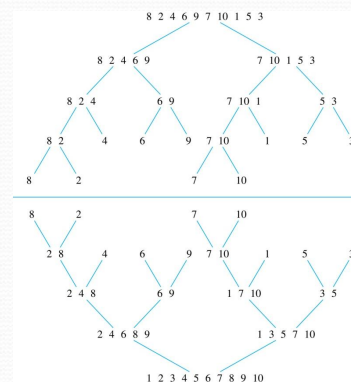
Merge Sort

Example: Use merge sort to put the list

8,2,4,6,9,7,10, 1, 5, 3

into increasing order.

Solution:



Recursive Merge Sort

Example: Construct a recursive merge sort algorithm.

Solution: Begin with the list of n elements L .

```

procedure mergesort( $L = a_1, a_2, \dots, a_n$ )
if  $n > 1$  then
     $m := \lfloor n/2 \rfloor$ 
     $L_1 := a_1, a_2, \dots, a_m$ 
     $L_2 := a_{m+1}, a_{m+2}, \dots, a_n$ 
     $L := \text{merge}(\text{mergesort}(L_1), \text{mergesort}(L_2))$ 
{ $L$  is now sorted into elements in increasing order}
  
```

continued →

Recursive Merge Sort

- Subroutine *merge*, which merges two sorted lists.

```

procedure merge( $L_1, L_2$  : sorted lists)
 $L :=$  empty list
while  $L_1$  and  $L_2$  are both nonempty
    remove smaller of first elements of  $L_1$  and  $L_2$  from its list;
    put at the right end of  $L$ 
if this removal makes one list empty
    then remove all elements from the other list and append them to  $L$ 
return  $L$  { $L$  is the merged list with the elements in increasing order}
  
```

Complexity of Merge: Two sorted lists with m elements and n elements can be merged into a sorted list using no more than $m + n - 1$ comparisons.

Merging Two Lists

Example: Merge the two lists 2,3,5,6 and 1,4.

Solution:

TABLE 1 Merging the Two Sorted Lists 2, 3, 5, 6 and 1, 4.

<i>First List</i>	<i>Second List</i>	<i>Merged List</i>	<i>Comparison</i>
2 3 5 6	1 4		$1 < 2$
2 3 5 6	4	1	$2 < 4$
3 5 6	4	1 2	$3 < 4$
5 6	4	1 2 3	$4 < 5$
5 6		1 2 3 4	
		1 2 3 4 5 6	

Complexity of Merge Sort

Complexity of Merge Sort: The number of comparisons needed to merge a list with n elements is $O(n \log n)$.

- For simplicity, assume that n is a power of 2, say 2^m .
- At the end of the splitting process, we have a binary tree with m levels, and 2^m lists with one element at level m .
- The merging process begins at level m with the pairs of 2^m lists with one element combined into 2^{m-1} lists of two elements. Each merger takes two one comparison.
- The procedure continues, at each level ($k = m, m-1, m-1, \dots, 3, 2, 1$) 2^k lists with 2^{m-k} elements are merged into 2^{k-1} lists, with 2^{m-k+1} elements at level $k-1$.
 - We know (by the complexity of the merge subroutine) that each merger takes at most $2^{m-k} + 2^{m-k} - 1 = 2^{m-k+1} - 1$ comparisons.

continued →

Complexity of Merge Sort

- Summing over the number of comparisons at each level, shows that

$$\sum_{k=1}^m 2^{k-1} (2^{m-k+1} - 1) = \sum_{k=1}^m 2^m - \sum_{k=1}^m 2^{k-1} = m2^m - (2^m - 1) = n \log n - n + 1,$$

because $m = \log n$ and $n = 2^m$.

(The expression $\sum_{k=1}^m 2^{k-1}$ in the formula above is evaluated as $2^m - 1$ using the formula for the sum of the terms of a geometric progression, from Section 2.4.)

- In Chapter 11, we'll see that the fastest comparison-based sorting algorithms have $O(n \log n)$ time complexity. So, merge sort achieves the best possible big- O estimate of time complexity.

Example: Give a recursive algorithm for Fibonacci Numbers

$$f_n = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ f_{n-1} + f_{n-2}, & \text{if } n \geq 2 \end{cases}$$

Algorithm : A Recursive Procedure for Fibonacci Numbers.

```

procedure fibonacci(n: nonnegative integer)
{
  if n = 0 then fibonacci(0) := 0
  else if n = 1 then fibonacci(1) := 1
  else fibonacci(n) := fibonacci(n-1) + fibonacci(n-2)
}
  
```

Algorithm An Iterative Procedure for Computing Fibonacci Numbers.

```

procedure iterative fibonacci(n: nonnegative integer)
  if n = 0 then y := 0
  else
    begin
      x := 0
      y := 1
      for i := 1 to n - 1
        begin
          z := x + y
          x := y
          y := z
        end
      end
    end
  {y is the nth Fibonacci number}

```

Recursive algorithm vs. iterative algorithm

For every recursive algorithm, there is an **equivalent** iterative algorithm.

Recursive algorithms are often **shorter, more elegant, and easier to understand** than their iterative counterparts.

However, iterative algorithms are usually **more efficient** in their use of space and time.

Homework

Sec. 5.4 29

Counting

Chapter 6

Chapter Summary

- The Basics of Counting (计数)
- The Pigeonhole Principle (鸽巢原理)
- Permutations and Combinations (排列与组合)
- Binomial Coefficients and Identities
- Generalized Permutations and Combinations
- Generating Permutations and Combinations

The Basics of Counting

Section 6.1

Section Summary

- The Product Rule
- The Sum Rule
- The Subtraction Rule
- The Division Rule
- Examples, Examples, and Examples
- Tree Diagrams

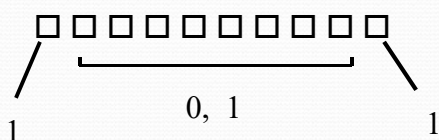
Basic Counting Principles: The Product Rule

The Product Rule: A procedure can be broken down into a sequence of two tasks. There are n_1 ways to do the first task and n_2 ways to do the second task. Then there are $n_1 \cdot n_2$ ways to do the procedure.

Example: How many bit strings of length seven are there?

Solution: Since each of the seven bits is either a 0 or a 1, the answer is $2^7 = 128$.

〔Example 〕 How many bit strings of length 10 begin and end with a 1?



Solution:

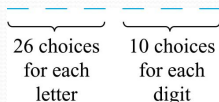
The product rule shows that there are a total of 2^8 different bit strings of length 10 begin and end with a 1.

27

The Product Rule

Example: How many different license plates can be made if each plate contains a sequence of three uppercase English letters followed by three digits?

Solution: By the product rule,
there are $26 \cdot 26 \cdot 26 \cdot 10 \cdot 10 \cdot 10 = 17,576,000$ different possible license plates.



Counting Functions

Counting Functions: How many functions are there from a set with m elements to a set with n elements?

Solution: Since a function represents a choice of one of the n elements of the codomain for each of the m elements in the domain, the product rule tells us that there are $n \cdot n \cdots n = n^m$ such functions.

Counting One-to-One Functions: How many one-to-one functions are there from a set with m elements to one with n elements? $m \leq n$

Solution: Suppose the elements in the domain are $a_1, a_2, \dots, a_m$. There are n ways to choose the value of a_1 and $n-1$ ways to choose a_2 , etc. The product rule tells us that there are $n(n-1)(n-2)\cdots(n-m+1)$ such functions.

Telephone Numbering Plan

Example: The *North American numbering plan (NANP)* specifies that a telephone number consists of 10 digits, consisting of a three-digit area code, a three-digit office code, and a four-digit station code. There are some restrictions on the digits.

- Let X denote a digit from 0 through 9.
- Let N denote a digit from 2 through 9.
- Let Y denote a digit that is 0 or 1.
- In the old plan (in use in the 1960s) the format was $NYX-NNX-XXXX$.
- In the new plan, the format is $NXX-NXX-XXXX$.

How many different telephone numbers are possible under the old plan and the new plan?

Solution: Use the Product Rule.

- There are $8 \cdot 2 \cdot 10 = 160$ area codes with the format NYX .
- There are $8 \cdot 10 \cdot 10 = 800$ area codes with the format NXX .
- There are $8 \cdot 8 \cdot 10 = 640$ office codes with the format NNX .
- There are $10 \cdot 10 \cdot 10 \cdot 10 = 10,000$ station codes with the format $XXXX$.

Number of old plan telephone numbers: $160 \cdot 640 \cdot 10,000 = 1,024,000,000$.

Number of new plan telephone numbers: $800 \cdot 800 \cdot 10,000 = 6,400,000,000$.

Counting Subsets of a Finite Set

Counting Subsets of a Finite Set: Use the product rule to show that the number of different subsets of a finite set S is $2^{|S|}$. (In Section 5.1, mathematical induction was used to prove this same result.)

Solution: When the elements of S are listed in an arbitrary order, there is a one-to-one correspondence between subsets of S and bit strings of length $|S|$. When the i th element is in the subset, the bit string has a 1 in the i th position and a 0 otherwise.

By the product rule, there are $2^{|S|}$ such bit strings, and therefore $2^{|S|}$ subsets.

Product Rule in Terms of Sets

- If $A_1, A_2, \dots, A_m$ are finite sets, then the number of elements in the Cartesian product of these sets is the product of the number of elements of each set.
- The task of choosing an element in the Cartesian product $A_1 \times A_2 \times \dots \times A_m$ is done by choosing an element in A_1 , an element in A_2 , ..., and an element in A_m .
- By the product rule, it follows that:

$$|A_1 \times A_2 \times \dots \times A_m| = |A_1| \cdot |A_2| \cdot \dots \cdot |A_m|.$$

DNA and Genomes

- A *gene* is a segment of a DNA molecule that encodes a particular protein and the entirety of genetic information of an organism is called its *genome*.
- DNA molecules consist of two strands of blocks known as nucleotides. Each nucleotide is composed of bases: adenine (A), cytosine (C), guanine (G), or thymine (T).
- The DNA of bacteria has between 10^5 and 10^7 links (one of the four bases). Mammals have between 10^8 and 10^{10} links. So, by the product rule there are at least 4^{10^5} different sequences of bases in the DNA of bacteria and 4^{10^8} different sequences of bases in the DNA of mammals.
- The human genome includes approximately 23,000 genes, each with 1,000 or more links.
- Biologists, mathematicians, and computer scientists all work on determining the DNA sequence (genome) of different organisms.

Basic Counting Principles: The Sum Rule

The Sum Rule: If a task can be done either in one of n_1 ways or in one of n_2 ways to do the second task, where none of the set of n_1 ways is the same as any of the n_2 ways, then there are $n_1 + n_2$ ways to do the task.

Example: The mathematics department must choose either a student or a faculty member as a representative for a university committee. How many choices are there for this representative if there are 37 members of the mathematics faculty and 83 mathematics majors and no one is both a faculty member and a student.

Solution: By the sum rule it follows that there are $37 + 83 = 120$ possible ways to pick a representative.

【Example】 Counting the number of elements in A ,
 $A = \{\text{length 10 bit strings with 0-streak of length exactly 5}\}.$

Solution:

Since the set A can be break up into the following case.

$$A_1 = \{000001****\} \quad (* \text{ is either 0 or 1})$$

$$A_2 = \{1000001***\}$$

$$A_3 = \{*1000001**\}$$

$$A_4 = \{**1000001*\}$$

$$A_5 = \{***1000001\}$$

$$A_6 = \{****100000\}$$

Apply the sum rule:

$$|A| = |A_1| + |A_2| + |A_3| + |A_4| + |A_5| + |A_6|$$

35

The Sum Rule in terms of sets.

- The sum rule can be phrased in terms of sets.
 $|A \cup B| = |A| + |B|$ as long as A and B are disjoint sets.

- Or more generally,

$$|A_1 \cup A_2 \cup \cdots \cup A_m| = |A_1| + |A_2| + \cdots + |A_m|$$

when $A_i \cap A_j = \emptyset$ for all i, j .

- The case where the sets have elements in common will be discussed when we consider the subtraction rule and taken up fully in Chapter 8.

Combining the Sum and Product Rule

Example: Suppose statement labels in a programming language can be either a single letter or a letter followed by a digit. Find the number of possible labels.

Solution: Use the product rule.

$$26 + 26 \cdot 10 = 286$$

Counting Passwords

- Combining the sum and product rule allows us to solve more complex problems.

Example: Each user on a computer system has a password, which is six to eight characters long, where each character is an uppercase letter or a digit. Each password must contain at least one digit. How many possible passwords are there?

Solution: Let P be the total number of passwords, and let P_6 , P_7 , and P_8 be the passwords of length 6, 7, and 8.

- By the sum rule $P = P_6 + P_7 + P_8$.
- To find each of P_6 , P_7 , and P_8 , we find the number of passwords of the specified length composed of letters and digits and subtract the number composed only of letters. We find that:

$$P_6 = 36^6 - 26^6 = 2,176,782,336 - 308,915,776 = 1,867,866,560.$$

$$P_7 = 36^7 - 26^7 =$$

$$78,364,164,096 - 8,031,810,176 = 70,332,353,920.$$

$$P_8 = 36^8 - 26^8 =$$

$$2,821,109,907,456 - 208,827,064,576 = 2,612,282,842,880.$$

Consequently, $P = P_6 + P_7 + P_8 = 2,684,483,063,360$.

[[Example]] Choose three different numbers from the integers between 1 to 300 such that the sum of the three integers can be divisible by 3. How many the ways are there?

Solution:

$$A = \{x \mid 1 \leq x \leq 300, x(\bmod 3) = 1\}$$

$$B = \{x \mid 1 \leq x \leq 300, x(\bmod 3) = 2\}$$

$$C = \{x \mid 1 \leq x \leq 300, x(\bmod 3) = 0\}$$

$$|A| = |B| = |C| = 100$$

- (1) All of the three numbers are chosen from the set A C_{100}^3
- (2) All of the three numbers are chosen from the set B C_{100}^3
- (3) All of the three numbers are chosen from the set C C_{100}^3
- (4) Chose one number from the set A, B, C $C_{100}^1 \times C_{100}^1 \times C_{100}^1$

39

Internet Addresses

- Version 4 of the Internet Protocol (IPv4) uses 32 bits.

Bit Number	0	1	2	3	4	8	16	24	31		
Class A	0	netid					hostid				
Class B	1	0	netid					hostid			
Class C	1	1	0	netid					hostid		
Class D	1	1	1	0	Multicast Address						
Class E	1	1	1	1	0	Address					

- Class A Addresses:** used for the largest networks, a 0, followed by a 7-bit netid and a 24-bit hostid.
- Class B Addresses:** used for the medium-sized networks, a 10, followed by a 14-bit netid and a 16-bit hostid.
- Class C Addresses:** used for the smallest networks, a 110, followed by a 21-bit netid and a 8-bit hostid.
 - Neither Class D nor Class E addresses are assigned as the address of a computer on the internet. Only Classes A, B, and C are available.
 - 1111111 is not available as the netid of a Class A network.
 - Hostids consisting of all 0s and all 1s are not available in any network.

Counting Internet Addresses

Example: How many different IPv4 addresses are available for computers on the internet?

Solution: Use both the sum and the product rule. Let x be the number of available addresses, and let x_A , x_B , and x_C denote the number of addresses for the respective classes.

- To find, x_A : $2^7 - 1 = 127$ netids. $2^{24} - 2 = 16,777,214$ hostids.
 $x_A = 127 \cdot 16,777,214 = 2,130,706,178$.
- To find, x_B : $2^{14} = 16,384$ netids. $2^{16} - 2 = 16,534$ hostids.
 $x_B = 16,384 \cdot 16,534 = 1,073,709,056$.
- To find, x_C : $2^{21} = 2,097,152$ netids. $2^8 - 2 = 254$ hostids.
 $x_C = 2,097,152 \cdot 254 = 532,676,608$.
- Hence, the total number of available IPv4 addresses is

$$\begin{aligned} x &= x_A + x_B + x_C \\ &= 2,130,706,178 + 1,073,709,056 + 532,676,608 \\ &= 3,737,091,842. \end{aligned}$$

Not Enough Today !!

The newer IPv6 protocol solves the problem of too few addresses. uses 128 bits

Basic Counting Principles: Subtraction Rule

Subtraction Rule: If a task can be done either in one of n_1 ways or in one of n_2 ways, then the total number of ways to do the task is $n_1 + n_2$ **minus** the number of ways to do the task that are common to the two different ways.

- Also known as, the *principle of inclusion-exclusion*:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

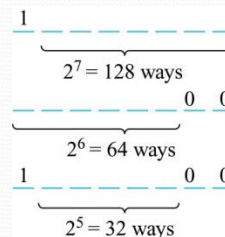
Counting Bit Strings

Example: How many bit strings of length eight either start with a 1 bit or end with the two bits 00?

Solution: Use the subtraction rule.

- Number of bit strings of length eight that start with a 1 bit: $2^7 = 128$
- Number of bit strings of length eight that start with bits 00: $2^6 = 64$
- Number of bit strings of length eight that start with a 1 bit and end with bits 00 : $2^5 = 32$

Hence, the number is $128 + 64 - 32 = 160$.



Basic Counting Principles: Division Rule

Division Rule: There are n/d ways to do a task if it can be done using a procedure that can be carried out in n ways, and for every way w , exactly d of the n ways correspond to way w .

- Restated in terms of sets: If the finite set A is the union of n pairwise disjoint subsets each with d elements, then $n = |A|/d$.
- In terms of functions: If f is a function from A to B , where both are finite sets, and for every value $y \in B$ there are exactly d values $x \in A$ such that $f(x) = y$, then $|B| = |A|/d$.

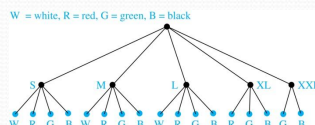
Example: How many ways are there to seat four people around a circular table, where two seatings are considered the same when each person has the same left and right neighbor?

Solution: Number the seats around the table from 1 to 4 proceeding clockwise. There are four ways to select the person for seat 1, 3 for seat 2, 2, for seat 3, and one way for seat 4. Thus there are $4! = 24$ ways to order the four people. But since two seatings are the same when each person has the same left and right neighbor, for every choice for seat 1, we get the same seating.

Therefore, by the division rule, there are $24/4 = 6$ different seating arrangements.

Tree Diagrams

- **Tree Diagrams:** We can solve many counting problems through the use of *tree diagrams*, where a branch represents a possible choice and the leaves represent possible outcomes.
- **Example:** Suppose that “I Love Discrete Math” T-shirts come in five different sizes: S,M,L,XL, and XXL. Each size comes in four colors (white, red, green, and black), except XL, which comes only in red, green, and black, and XXL, which comes only in green and black. What is the minimum number of stores that the campus book store needs to stock to have one of each size and color available?
- **Solution:** Draw the tree diagram.



- The store must stock 17 T-shirts.

Homework

Sec. 6.1 41, 58 ,70

The Pigeonhole Principle

Section 6.2

Section Summary

- The Pigeonhole Principle
- The Generalized Pigeonhole Principle

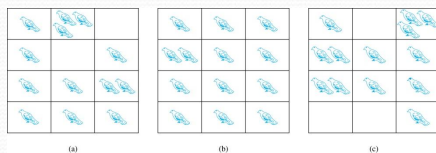
The Pigeonhole Principle

- Suppose a flock of pigeons fly into a set of pigeonholes to roost
- If there are more pigeons than pigeonholes, then there must be at least 1 pigeonhole that has more than one pigeon in it
- Main numbers
 - (1) the number of pigeon
 - (2) the number of pigeonhole

It is also called
Dirichlet Drawer Principle

The Pigeonhole Principle

- If a flock of 20 pigeons roosts in a set of 19 pigeonholes, one of the pigeonholes must have more than 1 pigeon.



Pigeonhole Principle: If k is a positive integer and $k + 1$ objects are placed into k boxes, then at least one box contains two or more objects.

Proof: We use a proof by contraposition. Suppose none of the k boxes has more than one object. Then the total number of objects would be at most k . This contradicts the statement that we have $k + 1$ objects. ◀

The Pigeonhole Principle

Corollary 1: A function f from a set with $k + 1$ elements to a set with k elements is not one-to-one.

Proof: Use the pigeonhole principle.

- Create a box for each element y in the codomain of f .
- Put in the box for y all of the elements x from the domain such that $f(x) = y$.
- Because there are $k + 1$ elements and only k boxes, at least one box has two or more elements.

Hence, f can't be one-to-one. ◀

Pigeonhole Principle

Example: Among any group of 367 people, there must be at least two with the same birthday, because there are only 366 possible birthdays.

Example (optional): Show that for every integer n there is a multiple of n that has only 0s and 1s in its decimal expansion.

Solution: Let n be a positive integer. Consider the $n + 1$ integers 1, 11, 111, ..., 11...1 (where the last has $n + 1$ 1s). There are n possible remainders when an integer is divided by n . By the pigeonhole principle, when each of the $n + 1$ integers is divided by n , at least two must have the same remainder. Subtract the smaller from the larger and the result is a multiple of n that has only 0s and 1s in its decimal expansion.

The direct applications of the pigeonhole principle

(1) Among any group of 11 integers, there are two integers a and b such that $10 \mid a-b$.

Pigeons : 11 integers

Pigeonholes: the possible remainders when an integer is divided by 10

(2) In a party of 2 or more people, there are 2 people with the same number of friends in the party. (Assuming you can't be your own friend and that friendship is mutual.)

Pigeons : the n people (with $n > 1$).

Pigeonholes: the possible number of friends, i.e.

the set $\{0, 1, 2, 3, \dots, n-1\}$

(注意: 0和n-1的情况不会同时出现)

53

The Generalized Pigeonhole Principle

The Generalized Pigeonhole Principle: If N objects are placed into k boxes, then there is at least one box containing at least $\lceil N/k \rceil$ objects.

Proof: We use a proof by contraposition. Suppose that none of the boxes contains more than $\lceil N/k \rceil - 1$ objects. Then the total number of objects is at most

$$k \left(\left\lceil \frac{N}{k} \right\rceil - 1 \right) < k \left(\left(\frac{N}{k} + 1 \right) - 1 \right) = N,$$

where the inequality $\lceil N/k \rceil < N/k + 1$ has been used. This is a contradiction because there are a total of n objects. ◀

Example: Among 100 people there are at least $\lceil 100/12 \rceil = 9$ who were born in the same month.

The Generalized Pigeonhole Principle

Example: a) How many cards must be selected from a standard deck of 52 cards to guarantee that at least three cards of the same suit are chosen?

b) How many must be selected to guarantee that at least three hearts are selected?

Solution: a) We assume four boxes; one for each suit. Using the generalized pigeonhole principle, at least one box contains at least $\lceil N/4 \rceil$ cards. At least three cards of one suit are selected if $\lceil N/4 \rceil \geq 3$. The smallest integer N such that $\lceil N/4 \rceil \geq 3$ is $N = 2 \cdot 4 + 1 = 9$.

b) A deck contains 13 hearts and 39 cards which are not hearts. So, if we select 41 cards, we may have 39 cards which are not hearts along with 2 hearts. However, when we select 42 cards, we must have at least three hearts. (Note that the generalized pigeonhole principle is not used here.)

Example

- What is the least number of area codes needed to guarantee that 25 million phones in a state have distinct ten-digit telephone number?
- Solution: number form: NXX-NXX-XXXX, there are 8×10^6 different phone numbers of the form NXX-XXXX, hence, by generalized pigeonhole principle, at least $\lceil 25 \times 10^6 / 8 \times 10^6 \rceil = 4$ area codes

(Let X denote a digit from 0 through 9.
Let N denote a digit from 2 through 9.)

Elegant application

- [Example] during 30 days a baseball team plays at least one game a day, but no more than 45 games; show that there must be a period of some number of consecutive days during which the team must play exactly 14 games.
- Solution: a_j : the number of games played on j -th day
- $b_j = \sum_{i=1}^j a_i$ then $b_1, b_2, \dots, b_{30}$ are strictly increasing number.
- $c_j = b_j + 14$, then $15 \leq c_j \leq 45 + 14 = 59$, the 60 integer numbers $b_1, b_2, \dots, b_{30}, c_1, c_2, \dots, c_{30}$ are all less than or equal to 59, then two of these integers are equal, hence
- $b_i = c_j = b_j + 14$, it implies $a_{j+1} + a_{j+2} + \dots + a_i = 14$

[Example] Suppose that there are n arbitrary integers $x_1, x_2, \dots, x_n$. Show that there exist some consecutive integers such that the sum of these integers is the multiple of n .

Solution:

$$a_i = \sum_{k=1}^i x_k \quad (i = 1, 2, \dots, n)$$

$$(1) \quad \exists i \quad (n | a_i)$$

$$(2) \quad \neg \exists i \quad (n | a_i)$$

【Example】 Every sequence of n^2+1 distinct integers contains a subsequence of length $n + 1$ that is either strictly increasing or strictly decreasing.

Proof:

For example, $n=2$

1,2,3,4,5 ; 4,8,3,6,1; 1,4,5,3,2

Let the sequence be $a_1, a_2, \dots, a_{n^2+1}$

Associate (x_k, y_k) to the term a_k , where x_k is the length of the longest increasing subsequence starting at a_k , and y_k is the length of the longest decreasing subsequence starting at a_k

Suppose that there is no increasing or decreasing subsequence of length $n+1$. Then

$$1 \leq x_k \leq n \quad 1 \leq y_k \leq n$$

59

Hence there are $n \times n = n^2$ pairs (x_k, y_k) ,

Since $a_i \neq a_j$

It follows that

(1) $a_i < a_j$

(2) $a_i > a_j$

In either case there is a contradiction.

60

Hence there are $n \times n = n^2$ pairs (x_k, y_k) ,

Since there are $n^2 + 1$ a_k , By the pigeonhole principle, it follows that there exist terms a_i, a_j ($1 \leq i < j \leq n^2 + 1$) such that $(x_i, y_i) = (x_j, y_j)$

Since $a_i \neq a_j$

It follows that

(1) $a_i < a_j$

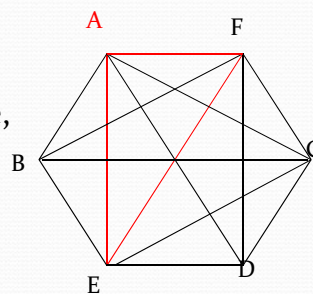
(2) $a_i > a_j$

In either case there is a contradiction.

61

Ramsey Theorem

- $K_6 \rightarrow K_3, K_3$, of 6 (or more) people, either there are 3, each pair of whom are acquainted, or there are three, each pair of whom are unacquainted.



Ramsey number $R(m, n)$

- The *Ramsey number* $R(m, n)$ is the smallest number of people at a party that there either m mutual friends or n mutual enemies. *e.g.*, $R(3, 3) = 6$
- $R(2, n) = R(n, 2) = n$
- 只知道9个拉姆齐数 $R(m, n)$ ($3 \leq m \leq n$) 的精确值

Homework

Sec. 6.2 12, 40, 42, 44